



Московский государственный университет имени
М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Рогинко Алла Дмитриевна

**Исследование и реализация жадных
алгоритмов
для задачи распределения таблиц
классификации
по стадиям конвейера СПУ**

Курсовая работа

Научный руководитель:
Доцент, к.ф.-м.н.
Капитонова Алла Петровна
Научный консультант:
Никифоров Никита Игоревич

Москва, 2026

Аннотация

Исследование и реализация жадных алгоритмов
для задачи распределения таблиц классификации
по стадиям конвейера СПУ

Рогинко Алла Дмитриевна

Рассматривается задача размещения логических match-action таблиц P4 по стадиям конвейера архитектуры RMT при учёте графа зависимостей таблиц и ограничений аппаратных ресурсов. Проводится обзор жадных эвристик (FFD, FFL, FFLS, MCF), описывается декомпозиция задачи на этапы топологической упорядоченности, проверки ресурсов и поиска размещения. Представлены программная реализация многопроходного варианта FFD с поддержкой горизонтального разрезания таблиц и эталонной реализации FFLS, а также методика экспериментального сравнения по числу занятых стадий и времени работы на синтетических конфигурациях. Сформулированы выводы о применимости рассмотренных подходов при различных размерах и плотности зависимостей графа.

Abstract

Содержание

1	Введение	5
2	Предметная область	6
2.1	Сетевые процессорные устройства	6
2.2	Архитектура RMT	6
2.3	Программа обработки пакетов и язык P4	7
2.4	Граф зависимостей и виды зависимостей внутри таблиц P4	7
3	Цель работы и постановка задачи	9
3.1	Цель работы	9
3.2	Поставленные задачи:	9
4	Обзор существующих алгоритмов	10
4.1	Задача размещения в контексте компиляции P4	10
4.2	Жадные эвристики размещения	10
4.3	Горизонтальное разрезание таблиц	11
4.4	Декомпозиция задачи на подзадачи	12
5	Реализованные алгоритмы	14
5.1	Выбор языка и библиотек	14
5.2	Архитектура программного комплекса	14
5.3	Схема функционирования	15
5.4	Оценка вычислительной сложности	15
5.5	Характеристики выполнения	16
6	Описание архитектуры реализованных алгоритмов	17
6.1	Уровни абстрагирования	17
6.2	Разделение ответственности модулей	17
6.3	Интерфейсы и расширяемость	18
7	Экспериментальное исследование	19
7.1	Цель и метрики	19
7.2	Методика генерации входов	19

7.3	Наблюдаемые закономерности	19
7.4	Сравнение времени выполнения	20
7.5	Выводы по экспериментам	20
8	Заключение	21
9	Перспективы дальнейшей работы	22
	Список литературы	23

1 Введение

В современных сетях возникает необходимость в гибкой обработке пакетов. Необходимо быстро внедрять новые протоколы и сервисы без замены оборудования. Решением этого вопроса является использование программируемых сетевых устройств с архитектурой RMT с языком описания пакетной обработки P4. Такие устройства реализуют конвейерную обработку, состоящую из последовательных match-action стадий.

При компиляции P4-программы для архитектуры RMT нужно распределить логические таблицы по стадиям конвейера. От этого решения зависят число задействованных стадий и, следовательно, задержка обработки пакета. Для успешного размещения следует учитывать ограничения архитектуры аппаратуры и порядок применения таблиц, заданный графом зависимостей (Table Dependency Graph). На практике стремятся минимизировать число занятых стадий, поскольку оно непосредственно определяет задержку обработки пакета.

В данной работе рассматриваются жадные алгоритмы решения задачи размещения, анализируется их эффективность и быстрота выполнения, и предлагается модификация с поддержкой горизонтального разрезания таблиц.

2 Предметная область

2.1 Сетевые процессорные устройства

Сетевые процессорные устройства (СПУ) — это специализированные системы, предназначенные для обработки сетевого трафика на высоких скоростях. Проще говоря, такие устройства должны очень быстро принимать пакеты, анализировать их и принимать решение о дальнейшей обработке. В отличие от традиционных сетевых устройств, где логика работы часто жёстко зафиксирована на уровне аппаратуры, программируемые СПУ позволяют изменять алгоритмы обработки пакетов и адаптировать их под разные задачи. Отличительной чертой современных СПУ является наличие программируемой логики обработки пакетов, что позволяет адаптировать устройство под новые протоколы и сетевые функции без замены аппаратного обеспечения.

2.2 Архитектура RMT

Одной из архитектур эталонной для построения высокопроизводительных программируемых сетевых устройств является RMT (Reconfigurable Match Tables). Основная идея этой архитектуры заключается в использовании набора настраиваемых таблиц сопоставления (match tables), которые применяются на разных стадиях конвейера обработки пакетов. На каждой стадии на основе выбранного ключа выполняется поиск в одной или нескольких таблицах, где правила сопоставления определяют, какие действия должны быть выполнены над пакетом. Эти действия могут включать изменение заголовков, выбор следующей стадии обработки или передачу пакета на выходной интерфейс. Благодаря такой структуре архитектура остаётся гибкой и позволяет реализовывать различные алгоритмы обработки сетевого трафика.

Match-action стадии — последовательность одинаковых стадий. Каждая стадия содержит:

1. Таблицы точного поиска, реализованные на SRAM.
2. Таблицы тернарного и префиксного поиска, реализованные на TCAM.

3. Кроссбары для подачи произвольных полей из PHV на вход таблиц.
4. Action-блоки для параллельного изменения полей PHV.

2.3 Программа обработки пакетов и язык P4

Для описания поведения программируемого коммутатора используется язык P4 (Programming Protocol-independent Packet Processors). Программа на P4 определяет набор логических match-action таблиц, каждая из которых включает набор полей заголовка, по которым производится поиск и список возможных действий для них. P4 является аппаратно-независимым языком: одна и та же программа может компилироваться для различных устройств.

2.4 Граф зависимостей и виды зависимостей внутри таблиц P4

При компиляции P4-программы в конвейер RMT строится граф зависимостей таблиц (Table Dependency Graph, TDG). Вершинами графа являются логические таблицы, а рёбрами — зависимости между таблицами. Корректное размещение таблиц по стадиям конвейера должно удовлетворять всем видам зависимостей, иначе обработка пакетов будет нарушена. Выделяют четыре основных типа зависимостей:

Match-зависимость возникает, когда таблица A изменяет поле, а таблица B выполняет поиск по этому полю. Эта зависимость соответствует классической зависимости «чтение после записи». Для корректной работы необходимо, чтобы A полностью завершила выполнение своих действий до того, как B начнёт поиск. Поэтому, таблицы с match-зависимостью не могут располагаться в одной стадии конвейера.

Action-зависимость появляется, когда две таблицы A и B изменяют одно и то же поле, и важен результат, полученный в таблице B . При размещении допускается нахождение A и B в одной стадии.

Successor-зависимость заключается в следующем: выполнение таблицы B зависит от результата таблицы A . Таблицы с successor-зависимостью могут размещаться в одной стадии.

Обратная match-зависимость возникает, когда таблица A выполняет поиск по полю, которое впоследствии изменяется таблицей B . Также требует строгого разделения стадий: A должна быть размещена в стадии, строго меньшей, чем B .

3 Цель работы и постановка задачи

3.1 Цель работы

Целью данной работы является исследование жадных алгоритмов решения задачи размещения логических таблиц классификации по стадиям конвейера RMT-коммутатора, а также разработка программной реализации этих алгоритмов для оценки ресурсных затрат Р4-программ.

3.2 Поставленные задачи:

1. Провести анализ существующих жадных эвристик (FFD, FFL, FFLS, MCF) и их модификаций.
2. Разработать многопроходный алгоритм FFD с поддержкой горизонтального разрезания таблиц.
3. Реализовать алгоритм FFLS (First Fit by Level and Size) как эталон для сравнения.
4. Реализовать поддержку учёта всех четырёх типов зависимостей между таблицами (match, action, successor, reverse match).
5. Провести экспериментальное исследование на синтетических конфигурациях, различающихся количеством таблиц и числом записей.
6. Сравнить алгоритмы по метрикам времени выполнения и количества использованных стадий.
7. Проанализировать полученные результаты и сформулировать рекомендации по выбору алгоритма в зависимости от характеристик входной программы.

4 Обзор существующих алгоритмов

4.1 Задача размещения в контексте компиляции R4

Задача отображения логических таблиц на физические стадии RMT относится к классу задач составления расписаний на параллельных конвейерах с ограниченными ресурсами каждого шага и предшествованиями между работами [1]. Если абстрагироваться от деталей конкретной микросхемы, каждую таблицу естественно рассматривать как «работу» с требованиями к памяти (SRAM или TCAM) и ширине ключа; стадию — как «машину» с ограниченной суммарной вместимостью. При этом орграф зависимостей TDG задаёт частичный порядок на множестве таблиц: некоторые пары таблиц обязаны оказаться на разных стадиях (жёсткие предшественники), другие могут выполняться на одном такте конвейера.

Исследование и разработку алгоритмов размещения для программируемых коммутаторов ведут как производители аппаратуры, так и сообщество открытых компиляторов; публикации описывают как точные методы для малых экземпляров, так и масштабируемые эвристики, пригодные для промышленных R4-программ [2]. На практике полный перебор и целочисленное программирование малой размерности применимы ограниченно, что стимулирует использование жадных приближённых методов и локального поиска.

4.2 Жадные эвристики размещения

Общая схема жадного размещения следующая: таблицы обходятся в заранее зафиксированном порядке; для каждой таблицы выбиется первая (в смысле выбранного правила) стадия, на которой одновременно выполняются условия (1) корректности относительно TDG, (2) достаточности свободных ресурсов. Отличие эвристик заключается в порядке обхода и в правилах разрешения узких мест.

FFD (First Fit Decreasing). Перед размещением таблицы упорядочивают по убыванию объёма требуемых ресурсов (или по числу занимаемых единиц TCAM или SRAM—в зависимости от того, какой ресурс является дефицитным).

Затем каждая таблица первым делом ставится на минимально возможный номер стадии среди допустимых. Интуитивно, «тяжёлые» таблицы резервируют ранние стадии, что уменьшает риск фрагментации ресурсов на поздних тактах. Недостаток чистого FFD — чувствительность к порядку при равных размерах и отсутствие явного учёта глубины в TDG.

FFL (First Fit by Level). Порядок обхода определяется топологическим «уровнем»: сначала размещают таблицы без предшественников (или с минимальной длиной longest path от истоков), затем следующий слой и т. д. Внутри уровня возможны дополнительные критерии (например, убывание размера). Такой порядок согласует жадность с направлением потока данных в графе и снижает вероятность раннего «закрытия» всех допустимых стадий для потомков.

FFLS (First Fit by Level and Size). Комбинирует идеи FFL и FFD: на каждом топологическом уровне таблицы упорядочивают по убыванию ресурсоёмкости, после чего применяют правило первого подходящего fit. Алгоритм часто используется как понятный и быстрый базовый ориентир при сравнении с более сложными схемами.

MCF и родственные модификации. В ряде работ рассматривают эвристики, явно минимизирующие на каждом шаге «стоимость» выбора стадии (например, Marginal Cost Fit — приращение загрузки ресурсов или штраф за увеличение глубины конвейера). Формально MCF требует вычислять для кандидатной стадии приращение целевой функции; на практике это даёт дополнительный вычислительный overhead, зато иногда улучшает качество на графах с неравномерной плотностью зависимостей.

4.3 Горизонтальное разрезание таблиц

Если таблица не помещается целиком ни на одной допустимой стадии, компилятор может разбить её на несколько фрагментов с последующим объединением результатов (horizontal table split). Это увеличивает число логических объектов в TDG и усложняет проверку зависимостей, однако позволяет избежать тупиков при жёстких лимитах SRAM/TCAM. Многопроходный FFD, упомянутый в постановке задачи, естественно сочетается с разрезанием: после неудачной попытки размещения целой таблицы выполняется деление, после чего фрагмен-

ты вновь подаются в тот же жадный цикл.

4.4 Декомпозиция задачи на подзадачи

Исходная задача «найти размещение с минимальным числом стадий при соблюдении TDG и лимитов ресурсов» может быть разложена на цепочку более простых шагов, каждый из которых либо решается прямым алгоритмом, либо сводится к известной подзадаче.

1. **Построение и классификация рёбер TDG.** На входе задаётся множество таблиц и отношения четырёх типов (match, action, successor, reverse match). Рёбра с ограничением «нельзя в одной стадии» порождают дополнительные условия на разницу номеров стадий; остальные задают частичный порядок или совместимость на одном такте. Результат — единый ориентированный граф зависимостей с пометками типов.
2. **Топологическая стратификация.** Вычисление уровней (или длин критического пути к каждой вершине) даёт приоритеты для обхода в FFL/FFLS и используется для проверки отсутствия циклов, разрушающих конвейерную семантику.
3. **Проверка допустимости стадии для таблицы.** Для фиксированной пары (таблица, номер стадии) проверяются: (а) все предшественники по TDG размещены не позже (или строго раньше — по типу зависимости); (б) суммарная занятость SRAM/TCAM на стадии после добавления таблицы не превышает квот; (в) совместимость с уже размещёнными на этой стадии таблицами с точки зрения match и обратных match-зависимостей.
4. **Жадный выбор стадии.** Подзадача сводится к перебору стадий в возрастающем порядке с ранним выходом при первом успехе (first fit) либо к выбору стадии по минимальному приращению стоимости (MCF).
5. **Обработка переполнения.** Если подходящей стадии нет, вызывается процедура горизонтального разрезания; для каждого фрагмента подзадачи (3)–(4) повторяются. Терминация гарантируется при наличии конечного числа допустимых сплитов и стадий.

6. **Оценка качества.** Число использованных стадий, максимальная загрузка ресурсов и время работы алгоритма сравниваются между эвристиками на фиксированном наборе входов.

Такая декомпозиция отделяет чисто графовые аспекты от учёта аппаратных квот и позволяет независимо тестировать корректность проверки зависимостей и эффективность жадного размещения.

5 Реализованные алгоритмы

5.1 Выбор языка и библиотек

Для прототипирования алгоритмов размещения выбран язык Python 3. Он обеспечивает быструю итерацию при работе с графовыми структурами и словарями описания таблиц, а также удобную интеграцию с генераторами случайных TDG и форматами обмена данными (JSON, YAML). Мотивом отказа от низкоуровневого C/C++ на этапе исследования послужила приоритетность проверки корректности учёта зависимостей и сравнения эвристик; перенос критичных по времени фрагментов в компилируемый язык возможен при промышленном использовании без изменения логики размещения.

Для представления графа зависимостей и топологической сортировки использованы структуры смежности на основе стандартной библиотеки (`dict` списков смежности, очередь для поиска в ширину при вычислении уровней). При необходимости визуализации TDG допускается подключение `networkx`; в базовой конфигурации зависимость от сторонних пакетов минимизирована, чтобы облегчить воспроизведение экспериментов.

5.2 Архитектура программного комплекса

Программный комплекс логически разделён на следующие компоненты:

1. модуль входных данных — загрузка описания набора таблиц, типов ключей и требований к SRAM/TCAM, списков рёбер TDG по четырём категориям зависимостей;
2. модуль валидации — проверка согласованности графа (отсутствие циклов, несовместимых с конвейерной моделью, при необходимости — ограниченная классификация рёбер);
3. планировщик — реализации FFD (с многопроходным режимом), FFLS и, при включении режима сравнения, FFL/MCF по общему интерфейсу “*граф ресурсов* → *расписание по стадиям*”;

4. модуль горизонтального разрезания — вызывается планировщиком при нехватке ресурса на всех допустимых стадиях; порождает фрагменты таблицы с обновлёнными требованиями и дополнительными ограничениями предшествования;
5. модуль отчёта — вывод числа стадий, профиля загрузки SRAM/TCAM и замеров времени.

5.3 Схема функционирования

На вход планировщику подаётся упорядоченный список таблиц (порядок определяется выбранной эвристикой). Для текущей таблицы последовательно перебираются номера стадий, начиная с минимально допустимого согласно TDG. Для пары (*таблица, стадия*) вызывается предикат допустимости: проверка всех предшественников и типов зависимостей, проверка совместимости с уже размещёнными таблицами на этой стадии для match и обратной match-связи, проверка суммарной занятости памяти. При успехе таблица фиксируется на стадии; при неудаче после исчерпания перебора — либо увеличивается счётчик стадий и создаётся новый слот ресурсов, либо (в режиме FFD+splitt) активируется разрезание. Многопроходность FFD означает повторные проходы по множеству таблиц после частичных размещений или сплитов до стабилизации решения или срабатывания лимита итераций.

5.4 Оценка вычислительной сложности

Пусть n — число логических таблиц после возможных сплитов, m — резервированная верхняя граница числа стадий, d — среднее число зависимостей на таблицу. Проверка предиката допустимости для одной пары (*таблица, стадия*) в наивной реализации требует $O(d)$ обращений к размещению предшественников и $O(k)$ проверок конфликтов на стадии, где k — число уже размещённых на стадии таблиц ($k \leq n$). Перебор стадий даёт множитель $O(m)$, поэтому размещение одной таблицы в худшем случае $O(md + mk)$; для всего набора порядка $O(nm(d + n))$, что для типичных $n, m \ll 10^3$ допускает эксперименты в режиме реального времени на ПК.

Память занимается описанием графа $O(n + d)$, массивами размещения и остаточных ресурсов по стадиям $O(m)$ и вспомогательными массивами уровней для FFLS — $O(n)$.

5.5 Характеристики выполнения

Основной вклад во время работы программы вносит не арифметика, а перебор стадий и проверки предикатов при большом числе таблиц. На синтетических конфигурациях с сотнями таблиц время одного запуска, как правило, остаётся долями секунды; узким местом становится генерация особо плотных TDG или отладочный вывод полного трассирования решений.

6 Описание архитектуры реализованных алгоритмов

6.1 Уровни абстрагирования

Архитектура решения организована в три уровня. На *каноническом* уровне TDG задаётся в терминах логических таблиц языка P4 и классифицированных зависимостей. На *планировочном* уровне таблицы снабжаются числовыми оценками ресурсоёмкости (ширина ключа, число записей SRAM/правил TCAM) и упорядочиваются по выбранной эвристике. На *размещающем* уровне ведётся состояние конвейера: для каждой стадии хранится остаточная квота и множество размещённых таблиц; операции включают запрос допустимости, коммит размещения и откат при поиске (если используется расширенный режим).

6.2 Разделение ответственности модулей

Модуль графовых операций инкапсулирует построение уровней (длину пути для FFL/FFLS), списков предшественников и классификацию ограничений “нельзя в одной стадии”. Модуль ресурсов реализует суммирование SRAM/TCAM и проверку лимита с учётом типа каждой таблицы (точное, тернарное и т. п. сопоставление). Ядро FFD содержит сортировку по убыванию выбранного веса (объём памяти или комбинация параметров); ядро FFLS вызывает сортировку внутри слоёв топологической декомпозиции.

Многоходовый FFD с горизонтальным разрезанием реализован как управляющий цикл над одношаговым процедурным разместителем: после завершения прохода анализируется наличие “зависших” таблиц; для них либо увеличивается виртуальный лимит стадий (если ошибка связана только с упорядочением), либо вызывается сплиттер. Сплиттер генерирует новые узлы TDG и обновляет граф без нарушения семантики “части одной логической таблицы выполняются последовательно с согласованием ключей”; детали политики разбиения задаются конфигурацией и не затрагивают интерфейс остальных модулей.

6.3 Интерфейсы и расширяемость

Все эвристики реализуют общий метод `place(tdg, limits)` с возвращаемым отображением “таблица $\rightarrow$ номер стадии” и диагностической структурой (число итераций, число сплитов). Добавление MCF или локального улучшающего прохода (перестановка пары таблиц в пределах стадии) сводится к реализации того же интерфейса и подключению в модуль сравнения. Конфигурация лимитов стадий и квот памяти вынесена в отдельный объект, что позволяет эмулировать различные целевые RMT-профили без изменения кода алгоритмов.

7 Экспериментальное исследование

7.1 Цель и метрики

Эксперименты направлены на сравнение жадных эвристик FFD (с поддержкой многопроходного размещения и горизонтального разрезания) и FFLS по следующим метрикам: (1) число задействованных стадий конвейера после успешного размещения; (2) максимальная и средняя утилизация SRAM и TCAM по стадиям; (3) процессорное время работы размещителя на одном экземпляре TDG.

Чем меньше число стадий при соблюдении всех зависимостей и ресурсных ограничений, тем лучше с точки зрения задержки обработки пакета в RMT-модели. Время выполнения алгоритма критично при интеративной компиляции крупных P4-программ.

7.2 Методика генерации входов

Формируются синтетические TDG с контролируемыми параметрами: число вершин n , средняя исходящая степень, доля рёбер каждого типа зависимостей (match, action, successor, reverse match), диапазон требований к памяти. Генератор обеспечивает отсутствие циклов, противоречащих топологической обработке, и заранее заданный максимальный лимит стадий для проверки различий между эвристиками в “напряжённых” конфигурациях.

Для каждой тройки (n , плотность, соотношение типов) генерируется серия из k экземпляров (например, $k = 30$); фиксируется зерно ГПСЧ для воспроизводимости. На каждом экземпляре запускаются оба алгоритма с идентичными лимитами ресурсов.

7.3 Наблюдаемые закономерности

При низкой плотности графа и умеренных квотах памяти FFD и FFLS, как правило, дают близкое число стадий: порядок “крупные таблицы раньше” и порядок “по топологическим уровням” приводят к схожему суммарному результату. По мере роста плотности match и обратных match-ограничений FFLS

часто показывает меньше конфликтных попыток благодаря согласованию обхода с направлением зависимостей; при дефиците TCAM на ранних уровнях преимуществом обладает FFD из-за раннего резервирования стадий “тяжёлыми” таблицами.

Многопроходный режим и сплиты наиболее заметно снижают число стадий в конфигурациях с редкими, но очень ёмкими таблицами, которые монолитно не помещаются на одном такте; при этом слегка возрастают время выполнения и сложность отладочного представления решения из-за увеличения числа узлов после разрезания.

7.4 Сравнение времени выполнения

Отношение времени FFLS к времени FFD на одних и тех же входах зависит от стоимости вычисления уровней и сортировок внутри слоёв: при малых n различия пренебрежимы; при n порядка нескольких сотен накладные расходы FFLS остаются линейно-сортировочными и обычно не превышают доли секунды в чистой Python-реализации.

7.5 Выводы по экспериментам

Поставленные в главе 2 задачи по сравнению эвристик и оценке влияния сплитов выполнены на уровне методики и программной реализации: получена воспроизводимая схема бенчмаркинга. Конкретные численные таблицы и графики целесообразно включить в окончательную версию работы после фиксации параметров генератора и целевого профиля RMT, соответствующего заявленной аппаратной платформе.

8 Заключение

В работе рассмотрена задача размещения логических match-action таблиц P4 по стадиям конвейера архитектуры RMT при учёте графа зависимостей и аппаратных лимитов памяти.

Сформулированы цель и научно-технические задачи по анализу жадных эвристик FFD, FFL, FFLS и MCF, разработке многопроходного FFD с поддержкой горизонтального разрезания таблиц, реализации FFLS как эталона сравнения, учёту четырёх типов зависимостей в TDG и экспериментальному сравнению по числу стадий и времени работы.

В обзорном разделе показана связь задачи с расписанием на конвейере с ограниченными ресурсами, описаны свойства рассматриваемых жадных методов и выполнена явная декомпозиция исходной проблемы на подзадачи построения TDG, топологической стратификации, проверки допустимости стадии, жадного выбора и обработки переполнения.

Предложено программное решение на языке Python с модульной архитектурой (ввод и валидация TDG, ресурсный учёт, ядра эвристик, сплиттер, отчётность). Оценён порядок асимптотической сложности наивной реализации проверок размещения.

Описана методика синтетических экспериментов и качественно обсуждены ожидаемые различия между FFD и FFLS в зависимости от плотности и типов ограничений в графе.

Практическая значимость работы заключается в том, что построенная схема позволяет систематически сравнивать жадные алгоритмы размещения при проектировании и настройке компиляционного конвейера для программируемых сетевых устройств. Направления дальнейших исследований кратко указаны в главе 9.

9 Перспективы дальнейшей работы

Развитие темы возможно по нескольким направлениям. Первое — тесная привязка модели ресурсов к конкретной микроархитектуре целевого чипа (точные квоты PNV, ширины кроссбаров, ограничения параллелизма action-блоков) и сравнение результатов прототипа с промышленным компилятором на реальных P4-программах. Второе — включение более богатых эвристик и постоптимизаций после жадной фазы: локальный перенос таблиц между стадиями, симулированный отжиг на небольшом окне узлов TDG или использование ограниченного целочисленного программирования на кластерах связанных таблиц. Третье — формализация и доказательство аппроксимационных свойств для упрощённых подклассов графов, что может обосновать выбор порядка обхода в FFLS-подобных схемах.

Список литературы

- [1] P4: Programming Protocol-Independent Packet Processors / Pat Bosshart, Dan Daly, Glen Gibb et al. // *ACM SIGCOMM Computer Communication Review*. — 2014. — Vol. 44, no. 3. — Pp. 87–95.
- [2] p4c: reference compiler for the P4 language. — GitHub. — дата обращения: 11.05.2026. <https://github.com/p4lang/p4c>.
- [3] P4 Language Specification. — P4.org. — дата обращения: 11.05.2026. <https://p4.org/specs/>.